



Digital Programmable Systems

Dipartimento di Ingegneria Elettrica e dell'Informazione
Corso di Laurea Magistrale in Ingegneria Informatica

Relazione Progettuale

S.C.A.L.E. PiggyBank

Smart Coin Allocation & Level Evaluator

Professore

Prof. Giuseppe Coviello

Studenti

Ivan Giacomo Colella – 601432

Giusy Carotenuto – 601402

INDICE

1. Introduzione.....	2
1.1 Stato dell'arte.....	2
1.2 Obiettivo del progetto.....	3
1.3 Specifiche di sistema.....	4
1.4 Il microcontrollore e il nucleo di elaborazione.....	6
1.5 Celle di carico.....	7
1.6 Il servo motore.....	8
2. Progettazione Hardware e Meccanica.....	9
2.1 Architettura generale.....	9
2.2 Metodo di fissaggio delle celle.....	9
2.3 Progettazione e realizzazione dello scivolo.....	9
2.4 Sistema di generazione delle vibrazioni.....	10
3. Elettronica del progetto.....	12
3.1 Celle di carico.....	13
3.2 Servomotore.....	15
4. Progettazione del Firmware (Il "core" del corso).....	18
4.1 Celle di carico.....	20
4.1.1 Collegamento tra il main e le celle di carico.....	23
4.2 Servomotore.....	24
4.2.1 Collegamento tra il main e il servomotore.....	26
4.3 Modulo Bluetooth: comunicazione wireless.....	27
4.3.1 Caratteristica COINS (Notifica).....	27
4.3.2 Caratteristica LIMIT (Scrittura/Lettura).....	28
4.3.3 Sincronizzazione con la Web App (Web Bluetooth).....	29
5. Risultati Sperimentali.....	30
5.1 Validazione sperimentale e Telemetria.....	30
5.2 Precisione gravimetrica e stabilità delle letture.....	30
5.3 Efficienza meccanica e revisione del circuito di pilotaggio.....	30
5.4 Stabilità della connessione Bluetooth Low Energy.....	31
6. Conclusioni e Sviluppi futuri.....	33
6.1 Conclusioni.....	33
6.2 Sviluppi futuri.....	33

1. Introduzione

Il progetto **S.C.A.L.E. PiggyBank** ha come obiettivo quello di sviluppare un salvadanaio intelligente e automatizzato basato sull'avanzato microcontrollore STM32WB09. Il sistema converte un oggetto tradizionalmente passivo in un nodo IoT *edge-computing*, integrando al suo interno l'acquisizione analogica ad altissima risoluzione da sensori di peso, l'elaborazione algoritmica embedded, la comunicazione wireless a bassissimo consumo tramite Bluetooth Low Energy (BLE 5.4) e un'attuazione meccanica controllata in anello chiuso per bloccare fisicamente l'accesso al raggiungimento di target preimpostati.

1.1 Stato dell'arte

Nel panorama dei sistemi per il riconoscimento e il conteggio automatico delle monete, si distinguono diverse famiglie tecnologiche, ciascuna con specifici compromessi tra costo, complessità e affidabilità. I sistemi ottici classificano i pezzi valutando diametro, colore e goffratura; pur essendo precisi, richiedono un'elaborazione dell'immagine che risulta computazionalmente proibitiva per un microcontrollore entry-level. I sistemi elettromagnetici analizzano la firma della lega metallica, garantendo un'eccellente selettività antifrode, ma necessitano di un'elettronica analogica dedicata estremamente complessa da calibrare.

Esistono poi due approcci più adatti a sistemi a basso costo e ad alta efficienza: i sistemi meccanici a fessure calibrate, che separano le monete sfruttando oggettivamente il loro diametro, e i sistemi gravimetrici, che ne misurano la massa nominale. Storicamente, i selettori puramente meccanici si rivelano estremamente robusti ed economici, ma sono del tutto privi della capacità nativa di tracciamento elettronico del saldo.

Per superare questo limite, il progetto S.C.A.L.E. PiggyBank propone un'innovativa architettura ibrida. Il sistema delega lo smistamento fisico alla geometria di **uno scivolo passivo**, che separa infallibilmente i tagli in base al diametro, e affida la digitalizzazione del dato a un sottosistema gravimetrico posto a valle. Questa sinergia garantisce un tracciamento elettronico preciso e scalabile, perfettamente adattabile alle risorse limitate di un'architettura MCU, senza la necessità di attuatori attivi per la separazione.

1.2 Obiettivo del progetto

L'obiettivo cardine del progetto è automatizzare la misurazione del contenuto monetario interfacciando un sistema di smistamento fisico a un ecosistema IoT moderno. Il ciclo di funzionamento si divide in due fasi sinergiche: lo scivolo meccanico sfrutta il diametro dei tagli monetari per canalizzarli in maniera oggettiva verso cinque scomparti fisicamente separati; successivamente, il microcontrollore subentra per la digitalizzazione del dato e il controllo degli accessi.

Le funzioni implementate dal sistema includono:

- **Acquisizione Parallela:** Lettura simultanea e indipendente del peso all'interno dei cinque scomparti tramite convertitori ADC a 24 bit (HX711) dedicati.
- **Elaborazione Dati:** Riconoscimento del numero esatto di monete presenti in ogni colonna (attraverso la divisione del peso netto per il peso specifico del taglio) e calcolo algoritmico del saldo totale in euro.
- **Connettività GATT:** Trasmissione periodica e asincrona del saldo via BLE (Bluetooth Low Energy) a una Web App, che funge da interfaccia utente.
- **Attuazione Meccanica di Blocco:** Ricezione via BLE di una soglia limite di risparmio configurata dall'utente. Il microcontrollore gestisce una macchina a stati che comanda in PWM l'angolo di un servomotore per interdire fisicamente l'accesso allo scivolo principale non appena il totale eguaglia o supera il target.

Un aspetto cruciale della release finale del progetto riguarda la netta separazione dei domini di potenza e di attuazione. Il motore DC, deputato alla generazione di vibrazioni meccaniche per fluidificare la discesa delle monete ed evitare inceppamenti nello scivolo, è stato elettricamente isolato dall'elettronica logica e dotato di un'alimentazione del tutto indipendente.

Di conseguenza, il servomotore PWM rappresenta l'unico attuatore controllato direttamente dalle periferiche della scheda STM32WB09, la cui attivazione risponde esclusivamente alla logica di superamento della soglia impostata via BLE.

1.3 Specifiche di sistema

Le caratteristiche operative, prestazionali e le scelte costruttive dell'ecosistema hardware e software S.C.A.L.E. PiggyBank riflettono un'ingegnerizzazione mirata a bilanciare la flessibilità della prototipazione con la robustezza della misurazione. Le specifiche sono declinate nei seguenti domini:

1. Core Computazionale e Connettività

- a. **Microcontrollore (MCU):** System-on-Chip (SoC) STM32WB09KEVx. Il package compatto VFQFPN32 a 20 I/O offre il giusto compromesso tra minimizzazione degli ingombri sul PCB e disponibilità di pin per la gestione parallela dei sensori.
- b. **Sottosistema Radio:** Stack Bluetooth Low Energy (BLE) 5.4. Il nodo opera in configurazione *GATT Server*, ottimizzando i consumi e garantendo una latenza minima nella trasmissione del payload dei dati.
- c. **Interfaccia Utente (UI):** Web App sviluppata in HTML/JavaScript, che sfrutta le API Web Bluetooth per una comunicazione asincrona, cross-platform e priva di applicazioni native intermedie.

2. Sottosistema di Misura (Gravimetrico)

- a. **Trasduttori: 5 celle di carico** a sbalzo (cantilever) in alluminio, ciascuna con una portata di targa di 1 kg.
Questa capacità di fondo scala garantisce un'eccellente risoluzione sul singolo taglio monetario (che pesa pochi grammi), offrendo al contempo un ampio margine contro i sovraccarichi meccanici accidentali e il peso cumulativo a scompartimento pieno.
- b. **Condizionamento del Segnale:** 5 convertitori dedicati **HX711** (ADC a 24 bit). L'assegnazione di un convertitore per ogni cella elimina il crosstalk (l'accoppiamento indesiderato di tipo elettromagnetico che si crea tra le tracce di un circuito stampato) e la latenza tipici dei multiplexer analogici.

3. Attuazione e Smistamento Meccanico

- a. **Attuatore di Interdizione:** Micro-servomotore PWM da hobby (modello **TG9e**).

Consente di bloccare il passaggio delle monete una volta raggiunto il limite impostato dall'utente



- b. **Sistema Vibrante di Smistamento:** Motore in corrente continua (6V), recuperato da componentistica per modellismo. Modificando l'albero motore con il calettamento eccentrico di un dado metallico, il componente è stato convertito in un attuatore ERM (*Eccentric Rotating Mass*). Questo genera un'onda meccanica ad alta frequenza che vince l'attrito statico delle monete nello scompartimento superiore. Il controllo è demandato a un interruttore hardware manuale (On/Off), mantenendo il sistema deterministico e svincolato dalla logica software del microcontrollore.

4. Infrastruttura Meccanica e di Potenza

- a. **Telaio e Smorzamento Acustico:** La struttura portante poggia su una base massiccia in legno spessa 5 cm. Questa scelta agisce da filtro meccanico passa-basso: l'elevata inerzia della base assorbe e smorza le vibrazioni prodotte dal motore ERM, impedendo che l'energia cinetica si accoppi alle celle di carico falsando la pesatura. Le pareti perimetrali e i divisori interni sono realizzati in legno compensato per ridurre il peso complessivo del volume.
- b. **Infrastruttura di Cablaggio:** La rete di interconnessione tra logica, front-end analogici (HX711) e attuatore di blocco è realizzata su **breadboard**. Tale approccio ha garantito la massima flessibilità topologica e rapidità d'intervento durante le delicate fasi di prototipazione, calibrazione dei sensori e debug dei segnali.
- c. **Topologia di Alimentazione:** Il sistema adotta un'architettura duale galvanicamente e fisicamente isolata, cruciale per l'integrità del segnale.
- Ramo di Potenza Sporca:* Un pacco batterie dedicato da 6 V (4 celle AA da 1,5 V in serie) alimenta esclusivamente il motore vibrante ERM.
 - Ramo Logico/Analogico:* Una singola batteria da 9 V alimenta l'elettronica sensibile (scheda STM32, HX711 e servomotore). Questa separazione

impedisce che i violenti picchi di assorbimento transitorio (*inrush current*) e il rumore elettromagnetico indotto dal motore DC provochino cadute di tensione sulla logica a 3.3 V o introducano rumore condotto sulle letture degli ADC a 24 bit.

1.4 Il microcontrollore e il nucleo di elaborazione

Il cuore del progetto è l'**STM32WB09KE** (primo della serie STM32WB0), un System-on-Chip (SoC) wireless ultra-low-power di STMicroelectronics. Questa scelta integra potenza di calcolo e ricetrasmittitore radio in un unico chip, riducendo drasticamente la distinta base (BOM): esternamente bastano solo un quarzo da 32 MHz e un balun per l'antenna.



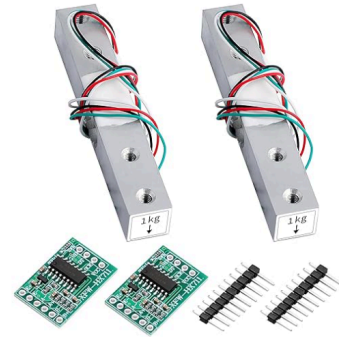
L'architettura interna risponde perfettamente alle esigenze del sistema tramite le seguenti caratteristiche chiave:

- **Calcolo e Ottimizzazione:** È basato su un core **Arm Cortex-M0+** (fino a 64 MHz), affiancato da 512 kbyte di Flash, 64 kbyte di SRAM e 1 kbyte OTP. Per massimizzare l'efficienza, la CPU legge i dati dalla memoria a 32 MHz (zero wait-state), mentre il timer avanzato a 16 bit sfrutta i 64 MHz per generare un segnale PWM ad altissima risoluzione per il servomotore.
- **Coprocessore Radio e BLE 5.4:** La radio è conforme allo standard Bluetooth LE 5.4 (supporta 1/2 Mbit/s e long-range). Il vero vantaggio è il **coprocessore radio con DMA**: gestisce in totale autonomia le rigide tempistiche del Bluetooth. Questo evita che la trasmissione dati "interrompa" o rallenti la delicata pesatura delle monete.
- **Consumi e Sicurezza:** Integra un convertitore step-down (SMPS) che prolunga al massimo la durata della batteria. Trattandosi di un dispositivo che custodisce risparmi, la sicurezza è garantita da hardware crittografico dedicato (PKA, TRNG, AES-128 e un ID univoco a 64 bit) per prevenire manomissioni digitali.
- **Interfacce e Formato:** Il package compatto **VFQFPN32** offre 20 pin di I/O, il numero esatto per interfacciare comodamente i cinque moduli HX711, il servo e le linee di

comunicazione (USART/LPUART, SPI, I2C), affiancati da un RTC low-power e un ADC a 12 bit.

1.5 Celle di carico

Il kit composto dal chip di conversione HX711 e da una cella di carico con portata fino a 1 kg rappresenta il cuore tecnologico per la realizzazione di sistemi di pesatura digitale ad alta precisione. Questa combinazione è progettata specificamente per tradurre variazioni fisiche microscopiche in dati digitali accurati, rivelandosi fondamentale nel mondo della prototipazione elettronica e dei sistemi embedded.



Ecco nel dettaglio come si articola il sistema:

1. L'Elettronica: Il convertitore ADC a 24 bit Il modulo HX711 non è un semplice sensore, ma un sofisticato **Convertitore Analogico-Digitale (ADC) a 24 bit** dotato di un amplificatore a guadagno programmabile (PGA) integrato.

- **Perché è importante?** Le celle di carico generano variazioni di tensione estremamente basse (spesso nell'ordine dei millivolt). L'HX711 amplifica questo debole segnale analogico e lo digitalizza con una risoluzione a 24 bit, garantendo una granularità di lettura altissima.
- **Stabilità e pulizia del segnale:** L'architettura del chip è studiata per ridurre al minimo il rumore elettrico e le interferenze. Questo assicura che i campionamenti avvengano in tempo reale con una stabilità costante, evitando le classiche "fluttuazioni" di peso anche in caso di utilizzo continuativo.

2. La Meccanica: La Cella di Carico Il componente che rileva fisicamente il peso è un solido blocco di alluminio a flessione.

- **Principio di funzionamento:** Al suo interno sfrutta il principio del *Ponte di Wheatstone*. Quando un oggetto viene posato sul piatto della bilancia, la struttura in alluminio subisce

una **deformazione elastica quasi impercettibile**. Questa flessione altera la resistenza elettrica degli estensimetri applicati sul metallo.

- **Portata e Sensibilità:** Essendo tarata per un carico massimo di **1 kg**, questa specifica cella offre il suo picco di sensibilità sulle misurazioni leggere e di precisione, rendendola la scelta d'elezione per bilance da banco, dosatori per alimenti o piccoli strumenti di laboratorio.

Interfacciamento: uno dei più grandi vantaggi del modulo HX711 è la semplicità di comunicazione. Utilizza un'interfaccia seriale a due soli fili (Data e Clock) che ne rende immediato l'utilizzo con qualsiasi microcontrollore moderno.

I PARAMETRI FONDAMENTALI:

1. **Uscita nominale ($1,0 \pm 0,15 \text{ mV/V}$):** Indica il segnale massimo che la cella invierà al modulo HX711 quando viene applicato il carico massimo (es. 1 kg). Questo valore aiuta a capire la sensibilità del sistema e influenza il "fattore di calibrazione";
2. **Tensione di eccitazione consigliata (5 V):**
3. **Protezione da sovraccarico (120 % RO) e Sovraccarico massimo (150% RO):**
Fissa i limiti meccanici.
4. **Dimensioni (12,7 mm x 12,7 mm x 80 mm).**

1.6 Il servo motore

Il Turnigy TG9e (spesso chiamato semplicemente TG9e) è un micro servo analogico della classe dei 9 grammi, famosissimo nel mondo dell'aeromodellismo.

I PARAMETRI FONDAMENTALI:

1. **Dimensioni:** 23 x 12,2 x 29 mm.
2. **Coppia (Torque):** 1,5 kg/cm (alimentato a 4.8V).
3. **Velocità operativa:** 0,12 sec/60° a 4.8V, e arriva a 0,10 sec/60° se alimentato a 6.0V.
4. **Elettronica e Motore:** È un servo analogico azionato da un classico motore a spazzole (Brushed).
5. **Banda morta (Dead band-width):** 7 μs .

2. Progettazione Hardware e Meccanica

Questo capitolo illustra la progettazione strutturale e le logiche meccaniche adottate per il progetto *S.C.A.L.E. PiggyBank*, descrivendo nel dettaglio come le scelte costruttive permettano al sistema di svolgere autonomamente la cernita e il conteggio del denaro in totale affidabilità.

2.1 Architettura generale

L'architettura generale del sistema si basa su uno scivolo inclinato a fori calibrati, progettato per smistare le monete in base al loro diametro.

A monte del sistema, un motorino vibrante fa defluire le monete, inserite alla rinfusa in un serbatoio iniziale, verso l'imbocco dello scivolo.

Qui, due paratie direzionali incanalano le monete, costringendole ad assumere il corretto allineamento per la discesa.

Sotto lo scivolo sono posizionati i contenitori di raccolta, ciascuno dei quali è interfacciato a una cella di carico per la misurazione del peso.

La disposizione dei fori segue la dimensione fisica e non il valore nominale: per questo motivo, la postazione dei 50 centesimi precede quella da 1 euro, avendo un diametro superiore.

Infine, non è previsto un foro dedicato alla moneta da 2 euro: essendo la più grande, quest'ultima prosegue lungo tutto il percorso per cadere per gravità direttamente nell'ultimo contenitore.

2.2 Metodo di fissaggio delle celle

La deformazione meccanica della cella di carico è resa possibile sollevandone la parte posteriore rispetto al piano di appoggio. Utilizzando un bullone come spessore, il sensore è stato fissato in modo sicuro alla base del progetto.

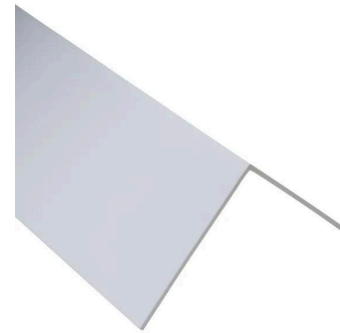
Sulla parte frontale, invece, è stato realizzato un supporto su misura per incastrare il bicchiere in cui cadono le monete. Il montaggio è stato curato per essere estremamente saldo, evitando così vibrazioni o instabilità che potrebbero falsare la misurazione.

2.3 Progettazione e realizzazione dello scivolo

Lo smistamento dimensionale passivo rappresenta il cuore della logica meccanica per la separazione oggettiva delle monete. La canalizzazione primaria è stata realizzata utilizzando un

profilo estruso a "L" in policloruro di vinile (PVC) rigido. Questo materiale è stato selezionato per la sua bassissima rugosità superficiale, una proprietà fondamentale per minimizzare il coefficiente di attrito radente e prevenire l'arresto accidentale della moneta lungo la discesa.

La geometria di separazione sfrutta una progressione di filtri passa-basso meccanici. Lungo la generatrice inferiore dello scivolo sono state fresate cinque feritoie a caduta, il cui interasse è perfettamente allineato con i sottostanti contenitori di raccolta. Le larghezze delle aperture (luci di passaggio) sono state calibrate in ordine strettamente crescente, rispecchiando il diametro nominale delle monete in euro: dalla più piccola (10 centesimi, 19.75 mm) a monte, fino alla più grande (2 euro, 25.75 mm) a valle, garantendo tolleranze sub-millimetriche.



La cinematica del sistema è stata ottimizzata empiricamente. L'angolo di inclinazione dello scivolo rispetto al piano orizzontale definisce l'accelerazione della moneta. In fase di taratura, la pendenza è stata regolata per imporre un tempo di volo di circa 2 secondi sull'intera lunghezza del canale. Questo smorzamento della dinamica è essenziale: riducendo l'energia cinetica orizzontale, si assicura che, nell'istante in cui il baricentro della moneta supera il limite di appoggio della feritoia corretta, la componente verticale della forza peso prevalga sull'inerzia orizzontale, garantendo una caduta fluida, deterministica e centrata nello scomparto di destinazione.

2.4 Sistema di generazione delle vibrazioni

Un limite intrinseco dei sistemi di smistamento ad alimentazione gravitazionale è il fenomeno del *jamming* (effetto imbuto) all'ingresso: l'incastro caotico di più monete che tentano di accedere simultaneamente alla sezione ristretta dello scivolo a causa dell'attrito statico. Per risolvere questa criticità e garantire un flusso in ingresso continuo e ordinato, il sistema combina un'attuazione vibrante con una geometria di filtri volumetrici a cascata.

Per quanto riguarda l'eccitazione meccanica, la tramoggia superiore è accoppiata a un motore in corrente continua (DC) a spazzole da 6 V. Tramite il calettamento di un dado metallico in

posizione eccentrica sull'albero motore, il componente è stato convertito in un attuatore ERM (*Eccentric Rotating Mass*). Questo trasmette sollecitazioni oscillatorie ad alta frequenza alla piattaforma, trasferendo alle monete l'energia cinetica necessaria a vincere l'attrito reciproco e fluidificare l'aggregato. Come discusso nell'architettura di sistema, per stringenti vincoli di compatibilità elettromagnetica (EMC), questo motore è alimentato da batterie dedicate ed è governato da un interruttore manuale isolato.

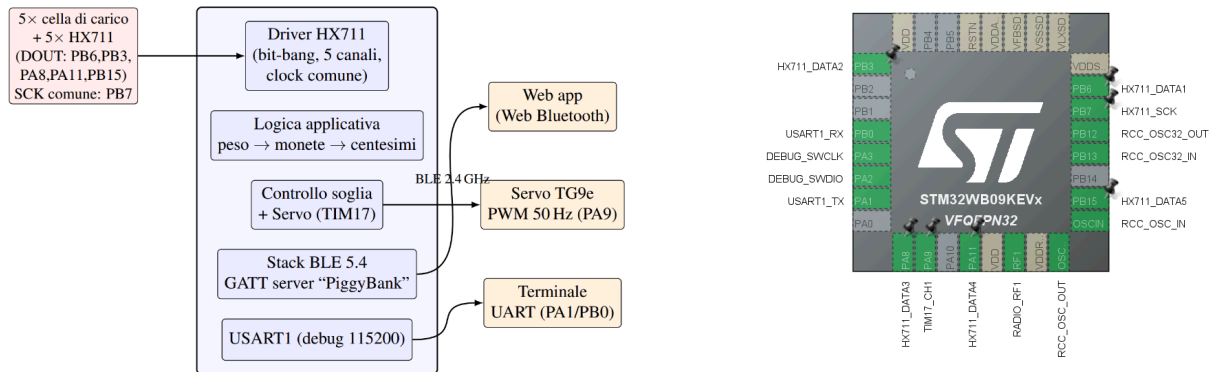


A supporto dell'azione vibrante, il design meccanico integra due feritoie di limitazione volumetrica (baffles) per preparare l'ingresso delle monete. Entrambe le aperture sono state calibrate con una tolleranza in altezza pari allo spessore massimo di una o due monete sovrapposte, in particolare abbiamo:

- La **prima feritoia**, posizionata all'interno della cabina superiore, funge da sbarramento primario: blocca il passaggio di ammassi caotici, consentendo il transito verso la zona di pre-carico solo a piccoli gruppi di monete parzialmente appiattite.
- La **seconda feritoia**, avente l'altezza di una sola moneta, è situata esattamente all'interfaccia tra la piattaforma superiore e l'inizio dello scivolo.

Questa architettura a doppio stadio è strategica: limita fisicamente lo spessore del materiale in transito mentre la vibrazione continua dell'ERM scompone le eventuali monete rimaste sovrapposte. L'azione sinergica di limitazione meccanica e scuotimento assicura che, superata la seconda feritoia, le monete siano perfettamente complanari e singolarizzate, pronte per affrontare il percorso di smistamento per diametro senza alcun rischio di inceppamento.

3. Elettronica del progetto

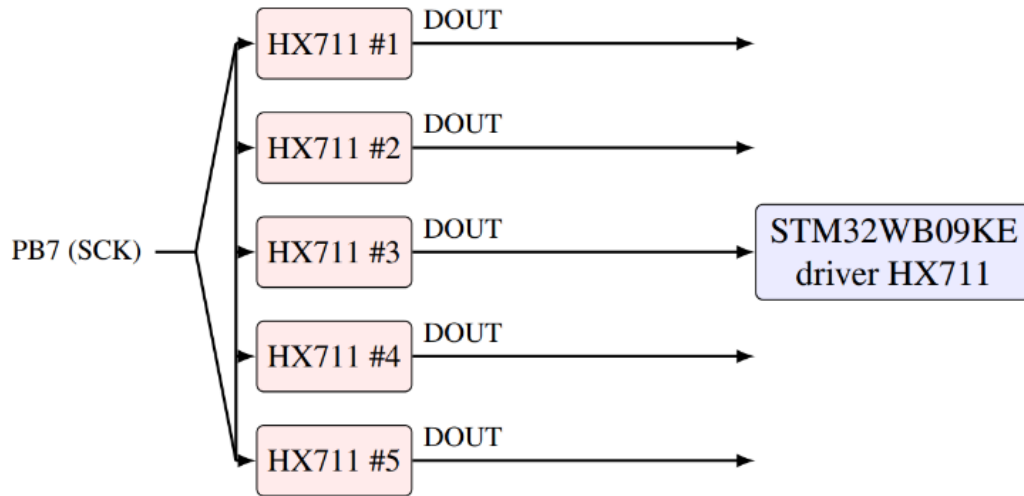


L'architettura è composta da quattro blocchi: **sensori di peso**, **microcontrollore STM32WB09**, **attuatore servo** e **interfaccia BLE/web app**.

Blocco	Elemento	Ruolo nel sistema
Sensori	5 celle di carico + 5 HX711	Misurano il peso associato ai cinque tagli di moneta.
Microcontrollore	STM32WB09KEVx	Legge gli HX711, conta le monete, gestisce BLE e PWM servo.
Attuatore	Servo tipo TG9e / microservo compatibile	Chiude meccanicamente il sistema quando il limite è raggiunto.
Comunicazione	Bluetooth Low Energy	Espone il servizio PiggyBank con caratteristiche COINS e LIMIT.
Interfaccia utente	Web app Web Bluetooth	Mostra monete/saldo e permette di scrivere il limite.

- Gli HX711 digitalizzano il segnale delle celle di carico e comunicano con il microcontrollore tramite protocollo sincrono bit-banging: il firmware genera gli impulsi SCK e legge i bit su DOUT;
- Lo STM32WB09 elabora i dati, gestisce il servizio BLE e genera il PWM per il servo.
- Il servo blocca il passaggio delle monete una volta raggiunto il limite fissato dall'utente attraverso interfaccia Bluetooth.

3.1 Celle di carico



Ogni scompartimento del salvadanaio utilizza una cella di carico collegata a un convertitore HX711.

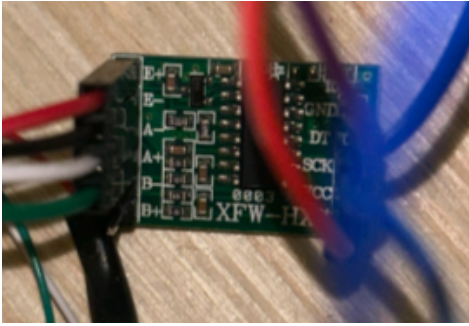
L’HX711 fornisce una misura digitale a 24 bit e segnala la disponibilità del nuovo dato portando la linea DOUT a livello basso (DOUT basso = dato pronto).

Nel progetto sono presenti cinque HX711: **tutti condividono il clock PB7**, mentre ogni modulo ha una propria linea dati.

La scelta del clock condiviso riduce il numero totale di pin: con 5 moduli servono 1 pin SCK + 5 pin DATA, invece di 10 linee separate.

Il firmware aspetta che tutte e cinque le linee dati siano basse prima di leggere una conversione completa. Il burst di lettura è composto da **24 impulsi per il dato e un 25° impulso finale che seleziona il canale A con guadagno 128 per la conversione successiva.**

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-d.	Maximum output sp.	Fast Mode	User Label	Modified
PA8	n/a	n/a	Input mode	No pull-up and no p.	n/a	n/a	HX711_DATA3	✓
PA11	n/a	n/a	Input mode	No pull-up and no p.	n/a	n/a	HX711_DATA4	✓
PB3	n/a	n/a	Input mode	No pull-up and no p.	n/a	n/a	HX711_DATA2	✓
PB6	n/a	n/a	Input mode	No pull-up and no p.	n/a	n/a	HX711_DATA1	✓
PB7	n/a	Low	Output Push Pull	No pull-up and no p.	Low	Disable	HX711_SCK	✓
PB15	n/a	n/a	Input mode	No pull-up and no p.	n/a	n/a	HX711_DATA5	✓



Connettori di ingresso: Rosso+ (positivo), Nero- (negativo);

Connettori di uscita: Verde+ (positivo), Bianco- (negativo).

Collegamenti: il rosso a E+, il nero a E-, il verde a A+ e il bianco a A-.

GPIO- GENERAL PURPOSE INPUT OUTPUT

Si tratta di un'interfaccia input/output su un circuito integrato che in fase di HW design non ha ancora un compito predefinito.

Descrizione dei Parametri di Configurazione GPIO

1. **Pin Name** (Nome del Pin): Indica l'identificativo fisico del piedino sul microcontrollore (es. PA8, PB7). La lettera indica la "Porta" (Port A, Port B), seguita dal numero del pin.
2. **Signal on Pin** (Segnale sul Pin): Specifica se il pin è assegnato a una funzione hardware interna dedicata (come un timer o una linea seriale UART/I2C). Essendo questi pin configurati come I/O generici (GPIO), il valore riportato è n/a (non applicabile).
3. **GPIO output level** (Livello di uscita iniziale): Definisce lo stato logico in cui si troverà il pin di output non appena il microcontrollore viene avviato, prima ancora che il tuo codice venga eseguito.

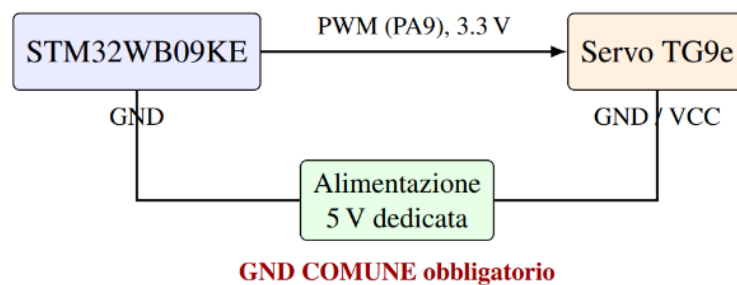
Nel caso del pin PB7 (il Clock), è impostato su Low (Basso, 0V). Per i pin di input questo parametro è ignorato (n/a). Livello iniziale posto basso prima della configurazione, così l'HX711 non riceve impulsi spuri all'avvio (un fronte di SCK farebbe avanzare il registro a scorrimento).

4. **GPIO mode** (Modalità operativa del GPIO): Determina la direzione del flusso di dati.
 - a. **Input mode:** Il microcontrollore è "in ascolto" e legge il segnale proveniente dall'esterno. È la configurazione usata per leggere i bit inviati dai moduli (HX711_DATA).
 - b. **Output Push-Pull:** Il microcontrollore genera il segnale in modo attivo. La modalità "Push-Pull" significa che il pin è in grado di "spingere" la tensione verso

l'alto (VCC) per il livello logico 1, e "tirarla" verso il basso (GND) per il livello logico 0. È ideale per pilotare il segnale di temporizzazione verso il sensore (HX711_SCK).

5. **GPIO Pull-up/Pull-down** (Resistenze interne): Tutti i pin sono impostati su "No pull-up and no pull-down" (modalità Floating). Il modulo HX711 spinge già attivamente il segnale dati in alto o in basso, quindi non c'è bisogno che il microcontrollore forzi un livello di riposo.

3.2 Servomotore

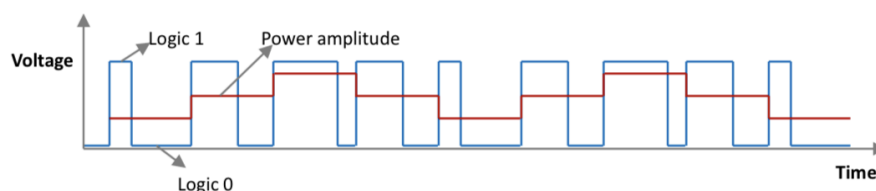


Il servo è comandato con un segnale PWM a periodo 20 ms, tipico dei servomotori hobby. Nel progetto il PWM è generato da **TIM17 CH1 sul pin PA9**.

Il PWM (dall'inglese Pulse Width Modulation, in italiano Modulazione a Larghezza di Impulso) è una tecnica utilizzata per controllare l'energia inviata a un dispositivo elettrico.

Invece di fornire una tensione continua e costante, il microcontrollore genera un segnale ad onda quadra che si "accende" e si "spegne" a una frequenza fissa.

La variabile che viene manipolata per controllare il dispositivo è il Duty Cycle.



I servomotori sfruttano questo treno di impulsi in ingresso per determinare l'**esatta posizione angolare del proprio albero**. A differenza di un normale motore DC in cui il PWM ne regola la velocità, nel servomotore la durata specifica dell'impulso controlla l'angolo di rotazione.

Il servo è dotato di 3 fili, che richiedono i seguenti collegamenti:

1. Rosso (Vin): Alimentazione, da collegare ai 5V.
2. Marrone (GND): Massa, da collegare a Terra.
3. Arancione/Giallo (Signal): Filo del segnale.

La configurazione del clock del microcontrollore influenza direttamente i parametri del segnale PWM generato. Il clock di sistema (**SYSCLK**) lavora a **64 MHz** e il segnale PWM è generato dal timer **TIM17** (Canale 1) sul pin PA9.

Per ottenere un **periodo compatibile con i servomotori (20 ms)**, il timer è stato configurato in questo modo:

1. **Prescaler (63)**: Divide la frequenza del sistema. La frequenza del contatore diventa **64 MHz / (63 + 1) = 1 MHz**.
2. **Tick del Timer**: Lavorando a 1 MHz, ogni scatto (tick) del timer equivale esattamente a **1 µs** (microsecondo). Questa scelta rende molto intuitivo il codice, permettendo di inserire il duty cycle direttamente in microsecondi.
3. **Periodo/ARR (19999)**: Poiché il conteggio del timer parte da 0, contando fino a 19999 si compiono **20000 tick**. Dato che ogni tick dura 1 µs, il periodo totale è di **20000 µs**, ovvero **20 ms**.
4. **Funzione `ServoTG9e_SetAngle()`**: Sfruttando la risoluzione a 1 µs, questa funzione converte i gradi desiderati nella corretta durata dell'impulso:
 - 0° corrisponde a un impulso di **500 µs**
 - 90° corrisponde a circa **1500 µs**
 - 180° corrisponde a **2500 µs**

$$f_{\text{tick}} = \frac{64\text{MHz}}{63+1} = 1\text{MHz} \Rightarrow T_{\text{tick}} = 1\mu\text{s}, \quad T_{\text{periodo}} = (19999+1) \cdot 1\mu\text{s} = 20\text{ms} \Rightarrow f_{\text{PWM}} = 50\text{Hz}.$$

4. Progettazione del Firmware (Il "core" del corso)

Il firmware è organizzato in codice generato da CubeMX e codice utente. Il file principale è Core/Src/main.c.

I driver custom sono Core/Src/hx711.c e Core/Src/servo_tg9e.c.

Il servizio BLE custom è implementato nei file STM32_BLE/App/piggybank.c e STM32_BLE/App/piggybank_app.c.

Parametri applicativi principali in main.c

```
/* USER CODE BEGIN PD */
#define HX711_TARE_SAMPLES      20U
#define HX711_AVERAGE_SAMPLES  5U
#define APP_STATUS_PERIOD_MS    500U
#define APP_BLE_NOTIFY_PERIOD_MS 500U
#define APP_UART_STATUS_ENABLE  1U
#define SERVO_OPEN_ANGLE        0U
#define SERVO_CLOSED_ANGLE      90U
```

Questi “define” fissano il comportamento temporale e meccanico: 20 campioni per la tara, 5 campioni per ogni media di peso, notifiche BLE e stampa UART ogni 500 ms, servo aperto a 0° e servo chiuso a 90°.

Variabili globali per pesi, conteggio monete e stato servo:

```
/* USER CODE BEGIN PV */
char messaggio_uart[180];
float pesi_reali[5] = {0.0f};
int monete_contate[5] = {0};
/* Peso nominale di una moneta per ogni canale fisico HX711.
   Ordine dei canali cablati: [0]=2€, [1]=50c, [2]=1€, [3]=20c, [4]=10c. */
const float PESI_MONETE[5] = {8.50f, 7.80f, 7.50f, 5.74f, 4.10f};
/* Stato logico del servo: 0 = aperto, 1 = chiuso.
   La chiusura avviene esclusivamente quando il totale raggiunge il limite BLE. */
volatile uint8_t g_servo_closed_by_limit = 0U;
/* USER CODE END PV */
```

IL CICLO while (1)

```
uint8_t hx711_tare_done = 0U;
uint32_t last_status_tick = 0U;
uint32_t last_ble_notify_tick = 0U;
while (1)
{
    uint32_t now = HAL_GetTick();
    App_HandleUartDebug();
    /* La tara viene eseguita in modo cooperativo: il BLE continua a essere
    processato. */
    if (hx711_tare_done == 0U)
    {
        if (HX711_TareAllTask(HX711_TARE_SAMPLES) != 0U)
        {
            hx711_tare_done = 1U;
            char tare_msg[] = "HX711 tare completed\r\n";
            HAL_UART_Transmit(&huart1, (uint8_t*)tare_msg, strlen(tare_msg),
50U);
        }
    }
    else
    {
        /* Ogni nuova media valida aggiorna conteggio monete, servo e notifica
        BLE. */
        if (HX711_GetWeightsAveragedTask(pesi_reali, HX711_AVERAGE_SAMPLES) !=
0U)
        {
            App_UpdateCoinCounts();
            App_UpdateServoFromLimit();
            if ((now - last_ble_notify_tick) >= APP_BLE_NOTIFY_PERIOD_MS)
            {
                last_ble_notify_tick = now;
                App_SendBleCoins();
            }
        }
        /* Stampa periodica su UART utile durante collaudo e calibrazione. */
        if ((now - last_status_tick) >= APP_STATUS_PERIOD_MS)
        {
            last_status_tick = now;
            App_PrintStatusUart();
        }
    }
}
```

```

    /* USER CODE END WHILE */
    MX_APPE_Process();
    App_UpdateServoFromLimit();
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

```

4.1 Celle di carico

Il driver HX711 è progettato per cinque convertitori in parallelo logico. Tutti usano la stessa linea SCK, ma ogni **convertitore ha il proprio DOUT**.

Il file `hx711.c` e le sue funzioni:

- La funzione **HX711_IsReadyAll()** restituisce 1 solo quando tutte le linee dati sono LOW. Questa scelta garantisce che una lettura simultanea produca un set coerente di cinque misure. `HX711_IsReadyAll()` ritorna 1 solo se tutte le cinque DOUT sono basse (l'HX711 segnala “pronto” abbassando DOUT). Richiedere che tutti i canali siano pronti sincronizza le letture su un istante comune. È non bloccante.

```

uint8_t HX711_IsReadyAll(void)
{
    return (
        (HAL_GPIO_ReadPin(HX711_DATA1_GPIO_Port, HX711_DATA1_Pin) ==
GPIO_PIN_RESET) &&
        (HAL_GPIO_ReadPin(HX711_DATA2_GPIO_Port, HX711_DATA2_Pin) ==
GPIO_PIN_RESET) &&
        (HAL_GPIO_ReadPin(HX711_DATA3_GPIO_Port, HX711_DATA3_Pin) ==
GPIO_PIN_RESET) &&
        (HAL_GPIO_ReadPin(HX711_DATA4_GPIO_Port, HX711_DATA4_Pin) ==
GPIO_PIN_RESET) &&
        (HAL_GPIO_ReadPin(HX711_DATA5_GPIO_Port, HX711_DATA5_Pin) ==
GPIO_PIN_RESET)
    ) ? 1U : 0U;
}

```

- **HX711_ReadReadyAll():** La lettura vera e propria è realizzata con bit-banging. Il firmware disabilita temporaneamente gli interrupt solo durante il breve burst di 24 bit, per evitare che il timing degli impulsi SCK venga disturbato. A ogni impulso il codice sposta

a sinistra i registri software e campiona le cinque linee DATA. Dopo 24 bit viene inviato un 25° impulso, usato dall'HX711 per selezionare canale A con guadagno 128 nella conversione successiva.

```
for (uint8_t i = 0; i < 24U; i++)
{
    HAL_GPIO_WritePin(HX711_SCK_GPIO_Port, HX711_SCK_Pin, GPIO_PIN_SET);
    __NOP(); __NOP(); __NOP(); __NOP();
    for(uint8_t j = 0; j < HX711_CHANNELS; j++)
    {
        data[j] <= 1;
    }
    HAL_GPIO_WritePin(HX711_SCK_GPIO_Port, HX711_SCK_Pin,
GPIO_PIN_RESET);
    __NOP(); __NOP(); __NOP(); __NOP();
    if (HAL_GPIO_ReadPin(HX711_DATA1_GPIO_Port, HX711_DATA1_Pin) ==
GPIO_PIN_SET) data[0]++;
    if (HAL_GPIO_ReadPin(HX711_DATA2_GPIO_Port, HX711_DATA2_Pin) ==
GPIO_PIN_SET) data[1]++;
    if (HAL_GPIO_ReadPin(HX711_DATA3_GPIO_Port, HX711_DATA3_Pin) ==
GPIO_PIN_SET) data[2]++;
    if (HAL_GPIO_ReadPin(HX711_DATA4_GPIO_Port, HX711_DATA4_Pin) ==
GPIO_PIN_SET) data[3]++;
    if (HAL_GPIO_ReadPin(HX711_DATA5_GPIO_Port, HX711_DATA5_Pin) ==
GPIO_PIN_SET) data[4]++;
}
/* 25° impulso: seleziona canale A con guadagno 128 per la lettura
successiva. */
HAL_GPIO_WritePin(HX711_SCK_GPIO_Port, HX711_SCK_Pin, GPIO_PIN_SET);
__NOP(); __NOP(); __NOP(); __NOP();
HAL_GPIO_WritePin(HX711_SCK_GPIO_Port, HX711_SCK_Pin, GPIO_PIN_RESET);
```

Il dato HX711 è un valore signed a 24 bit. Il bit 23 è il segno: se data & 0x800000, si estende con 0xFF000000 (complemento a due 24→32)

- **HX711_ReadAllIfReady()**: wrapper non bloccante (legge solo se pronto).

```
uint8_t HX711_ReadAllIfReady(int32_t* dati_uscita)
{
    if (HX711_IsReadyAll() == 0U)
    {
        return 0U;
    }
}
```

```

}
HX711_ReadReadyAll(dati_uscita);
return 1U;
}

```

- **HX711_TareAllBegin() / HX711_TareAllTask(times):** tara cooperativa. A ogni chiamata somma un campione pronto; dopo times campioni (20 nel nostro caso) calcola l'offset medio per canale e ritorna 1, spalmando la tara su più iterazioni senza bloccare il BLE.
- **HX711_SetScales(...):** fattori di scala da calibrazione; nel main.c sono impostati, uno per canale, ai valori 1050.0, 1044.5, 1007.0, 1073.9, 1017.39 (diversi perche' ogni cella ha sensibilita' leggermente diversa).
 - Per effettuare la tara di ogni cella di carico è stato utilizzato un oggetto target dal peso di 23g. Il valore mostrato a schermo è stato successivamente diviso per il peso del target, ottenendo in questo modo la scala specifica per ogni bilancia.
- **HX711_GetWeightsAveragedTask(pesi, times):** acquisizione cooperativa mediata: somma finche' non accumula times (nel nostro caso 5) campioni, poi converte:

$$\text{pesoj} = (\text{mediaj} - \text{offsetj}) / \text{scalaj}.$$
 - Con 5 campioni a 10 S il tempo per aggiornamento e' ~0.5s, coerente col periodo di notifica BLE.

Solo a scopo di debug e analisi, sono state lasciate disponibili le funzioni HX711_TareAll / HX711_GetWeights. Esse rappresentano le versioni bloccanti legacy, non usate nel main con BLE.

Le firme delle funzioni sono definite nel file **HX711.h**:

```

/*
 * hx711.h
 * Interfaccia per acquisizione pesi da cinque HX711.
 */
#ifndef INC_HX711_H_
#define INC_HX711_H_
#include "main.h"

```

```

// Prototipi delle funzioni per 5 bilance
void HX711_ReadAll(int32_t* dati_uscita);
void HX711_TareAll(uint8_t times);
void HX711_SetScales(float s1, float s2, float s3, float s4, float s5);
void HX711_GetWeights(float* pesi_uscita, uint8_t times);
// Funzioni non bloccanti quando BLE è attivo
uint8_t HX711_IsReadyAll(void);
uint8_t HX711_ReadAllIfReady(int32_t* dati_uscita);
void HX711_TareAllBegin(void);
uint8_t HX711_TareAllTask(uint8_t times);
uint8_t HX711_GetWeightsAveragedTask(float* pesi_uscita, uint8_t times);
#endif /* INC_HX711_H */

```

4.1.1 Collegamento tra il main e le celle di carico

La tara non viene eseguita con un'unica funzione bloccante nel main loop. All'avvio il firmware chiama HX711_TareAllBegin(); poi, a ogni ciclo, HX711_TareAllTask() prova ad acquisire un campione solo se tutti gli HX711 sono pronti. Quando sono stati raccolti 20 campioni, il driver calcola la media e memorizza gli offset. Questa architettura è importante perché il BLE deve continuare a chiamare MX_APPE_Process() e non può rimanere fermo in una lunga attesa.

Il vero calcolo delle monete:

- **App_UpdateCoinCounts:** dopo aver calcolato il peso netto, il firmware converte ogni peso in numero di monete dividendo per il peso nominale del taglio corrispondente e arrotondando.

È presente una banda morta di ± 2 g per evitare che rumore o piccole derive vengano interpretati come monete (2 g < metà della moneta più leggera, 4.1 g, quindi non nasconde mai una moneta reale); l'arrotondamento $(\text{int})(x+0.5)$ porta all'intero più vicino di monete.

```

static void App_UpdateCoinCounts(void)
{
    for (uint8_t i = 0U; i < 5U; i++)
    {
        if ((pesi_reali[i] > -2.0f) && (pesi_reali[i] < 2.0f))
        {
            pesi_reali[i] = 0.0f;
        }
    }
}

```



```

if (pesi_reali[i] <= 0.0f)
{
    monete_contate[i] = 0;
}
else
{
    monete_contate[i] = (int)((pesi_reali[i] / PESI_MONETE[i]) + 0.5f);
}
}
}

```

Indice array	Pin DATA	Taglio fisico	Peso nominale	Valore usato nel totale
0	PB6 / HX711_DATA1	2 euro	8.50 g	200 cent
1	PB3 / HX711_DATA2	50 cent	7.80 g	50 cent
2	PA8 / HX711_DATA3	1 euro	7.50 g	100 cent
3	PA11 / HX711_DATA4	20 cent	5.74 g	20 cent
4	PB15 / HX711_DATA5	10 cent	4.10 g	10 cent

- **App_GetTotalCents:**

```

static uint32_t App_GetTotalCents(void)
{
    uint32_t total = 0U;
    if (monete_contate[0] > 0) { total += (uint32_t)monete_contate[0] * 200U;
} /* channel 0 = 2€ */
    if (monete_contate[1] > 0) { total += (uint32_t)monete_contate[1] * 50U;
} /* channel 1 = 50c */
    if (monete_contate[2] > 0) { total += (uint32_t)monete_contate[2] * 100U;
} /* channel 2 = 1€ */
    if (monete_contate[3] > 0) { total += (uint32_t)monete_contate[3] * 20U;
} /* channel 3 = 20c */
    if (monete_contate[4] > 0) { total += (uint32_t)monete_contate[4] * 10U;
} /* channel 4 = 10c */
    return total;
}

```

4.2 Servomotore

Il servo viene inizializzato dopo la configurazione di TIM17 e prima dell'avvio dello stack BLE. Il firmware avvia il PWM su TIM17 CH1, imposta il servo aperto e mantiene una variabile di

stato `g_servo_closed_by_limit`. La chiusura è automatica: se `LIMIT > 0` e `totale >= LIMIT`, il servo passa all'angolo di chiusura; altrimenti resta o torna aperto.

Il file `servo_tg9e.c` e le sue funzioni:

- **ServoTG9e_Init (Funzione di Inizializzazione):** la funzione riceve come parametri il **puntatore alla struttura del timer** (es. `&htim17`) e il **canale su cui operare**.

Dopo aver salvato questi riferimenti nelle variabili statiche viste in precedenza, richiama la funzione nativa di libreria **HAL_TIM_PWM_Start()**. Questo comando "accende" fisicamente la periferica hardware del microcontrollore, facendo partire la generazione ininterrotta dell'onda quadra (il segnale PWM) sul pin associato.

- **ServoTG9e_SetAngle (Funzione di Controllo):** il primo blocco `if (angolo > 180U)` è un meccanismo di sicurezza. I servi hobbistici tipicamente non possono ruotare oltre i 180 gradi. Se il programma dovesse erroneamente richiedere, ad esempio, 200°, la funzione forza il valore a 180 (la U sta per Unsigned, intero senza segno) per prevenire lo stallo o il danneggiamento meccanico degli ingranaggi interni.

Successivamente trasforma il comando spaziale (gradi) nel comando temporale (microsecondi). Poiché lo 0° corrisponde a 500 µs e il 180° a 2500 µs, il range totale (Delta) è di 2000 µs.

Una volta calcolati i microsecondi necessari, viene invocata la macro **__HAL_TIM_SET_COMPARE**. Questa macro scrive il nuovo valore calcolato all'interno del registro **CCR (Capture/Compare Register)** del timer. Poiché l'hardware del timer è stato configurato a monte affinché 1 scatto del contatore (tick) equivalga a 1 µs, scrivere il valore calcolato modifica istantaneamente la larghezza dell'impulso generato, facendo ruotare il servomotore.

Le firme delle funzioni sono definite nel file **servo_tg9e.h**:

```
#ifndef SERVO_TG9E_H
#define SERVO_TG9E_H
#include "stm32wb0x_hal.h"
/* Inizializza il servo associandolo al timer/canale PWM scelto. */
void ServoTG9e_Init(TIM_HandleTypeDef *htim, uint32_t channel);
/* Imposta l'angolo del servo nell'intervallo 0..180 gradi. */
void ServoTG9e_SetAngle(uint8_t angolo);
```

```
#endif /* SERVO_TG9E_H */
```

4.2.1 Collegamento tra il main e il servomotore

- **App_UpdateServoFromLimit():** La funzione App_UpdateServoFromLimit() confronta il totale calcolato dal firmware con il limite letto dal modulo BLE. Il servo viene aggiornato solo se lo stato cambia, evitando di riscrivere continuamente il registro CCR del timer a ogni ciclo.

```
/* Aggiorna il servo solo quando cambia la condizione di soglia.  
Questo evita di riscrivere continuamente il PWM a ogni ciclo del main  
loop. */
```

```
static void App_UpdateServoFromLimit(void)
{
    static uint8_t last_closed_state = 0xFFU;
    uint32_t limit_cents = PIGGYBANK_APP_GetLimitValue();
    uint32_t total_cents = App_GetTotalCents();
    uint8_t should_close = ((limit_cents > 0U) && (total_cents >=
limit_cents)) ? 1U : 0U;
    if (should_close != last_closed_state)
    {
        if (should_close != 0U)
        {
            ServoTG9e_SetAngle(SERVO_CLOSED_ANGLE);
            APP_DBG_MSG("Servo CLOSED: total=%lu cents limit=%lu cents\r\n",
total_cents, limit_cents);
        }
        else
        {
            ServoTG9e_SetAngle(SERVO_OPEN_ANGLE);
            APP_DBG_MSG("Servo OPEN: total=%lu cents limit=%lu cents\r\n",
total_cents, limit_cents);
        }
        g_servo_closed_by_limit = should_close;
        last_closed_state = should_close;
    }
}
```

Condizione	Azione servo	Angolo
LIMIT = 0	Aperto: nessun blocco attivo	0°
LIMIT > 0 e totale < LIMIT	Aperto	0°
LIMIT > 0 e totale >= LIMIT	Chiuso	90°

4.3 Modulo Bluetooth: comunicazione wireless

La connettività IoT, pilastro del monitoraggio e dell'attuazione remota, è gestita dallo stack BLE 5.4 integrato nativamente nel SoC STM32WB09. Il firmware configura il dispositivo assumendo il ruolo di **GAP Peripheral** e **GATT Server**.

Profilo GAP e Advertising: Durante la fase di inizializzazione (`BLE_Init()`), vengono definiti i parametri del *Generic Access Profile* (GAP). Il dispositivo viene configurato come *General Discoverable* con *Appearance* impostata su `UNKNOWN`. Il pacchetto di *Advertising Data* espone in chiaro il nome completo tramite il flag `AD_TYPE_COMPLETE_LOCAL_NAME` con la stringa `"PiggyBank"`. Per garantire un accoppiamento rapido da parte del client, è stato impostato un intervallo di *Fast Advertising* con `ADV_INTERVAL_MIN` a `0x0080` (80 ms) e `ADV_INTERVAL_MAX` a `0x00A0` (100 ms). Una volta instaurata la connessione, il firmware negozia un *Connection Interval* di **1000 ms**, ottimizzando così i consumi energetici per una telemetria a bassa frequenza.

Architettura GATT: L'albero del *Generic Attribute Profile* (GATT) è strutturato attorno a un singolo servizio primario custom, identificato univocamente dall'UUID a 128 bit `12345678-1234-5678-1234-56789abcdef0`. All'interno di questo servizio sono state istanziate due caratteristiche proprietarie, gestite dalla funzione di callback `PIGGYBANK_EventHandler()`:

4.3.1 Caratteristica COINS (Notifica)

- **UUID:** `12345678-1234-5678-1234-56789abcdef1`
- **Proprietà:** `NOTIFY`
- **Lunghezza:** 5 byte + descrittore CCCD.

Questa caratteristica è deputata all'invio periodico (ogni **500 ms**) dei conteggi. Quando il client Web abilita le notifiche, lo stack genera l'evento `ATTRIBUTE_MODIFIED` sul CCCD. Il firmware elabora il buffer dei dati mediante la funzione `App_SendBleCoins()`, la quale

esegue un *riordino spaziale* degli indici: i canali fisici dell'hardware differiscono infatti dall'ordine di visualizzazione richiesto dall'interfaccia. L'array `coin_payload` viene così mappato:

- `coin_payload[0] = monete[0]` (Taglio 2€ - Canale fisico 0)
- `coin_payload[1] = monete[2]` (Taglio 1€ - Canale fisico 2)
- `coin_payload[2] = monete[1]` (Taglio 50c - Canale fisico 1)
- `coin_payload[3] = monete[3]` (Taglio 20c - Canale fisico 3)
- `coin_payload[4] = monete[4]` (Taglio 10c - Canale fisico 4)

Prima dell'assegnazione, ogni valore viene saturato a 8 bit (valore massimo 255) per prevenire l'overflow del buffer. La notifica `PIGGYBANK_Coins_SendNotification()` viene infine immessa in rete solo se esiste una connessione attiva (`ConnectionHandle != 0xFFFF`) e il client ha esplicitamente richiesto i dati.

4.3.2 Caratteristica LIMIT (Scrittura/Lettura)

- **UUID:** `12345678-1234-5678-1234-56789abcdef2`
- **Proprietà:** `READ | WRITE`
- **Lunghezza:** 4 byte.

Questa caratteristica riceve la soglia di blocco meccanico in formato intero a 32 bit senza segno (`uint32_t`), codificata in **Little-Endian**. Quando il client invia il comando GATT di scrittura, il modulo genera l'evento hardware `ACI_GATT_SRV_WRITE_VSEVT_CODE`, che scatena a livello applicativo l'evento `PIGGYBANK_LIMIT_WRITE_EVT`. All'interno dell'handler, i 4 byte del payload vengono estratti e ricostruiti matematicamente tramite bit-shifting:

```
LimitValue = p[0] | (p[1]<<8) | (p[2]<<16) | (p[3]<<24);
```

Questo intero rappresenta il limite rigidamente in centesimi di euro. A titolo di esempio, se l'utente imposta una soglia di **50.00 €**, il valore viene serializzato in **5000** centesimi (esadecimale

0x00001388); nello spazio di memoria *Little-Endian*, i byte fisici ricevuti risulteranno allocati come 88 13 00 00.

4.3.3 Sincronizzazione con la Web App (Web Bluetooth)

Per l'interfaccia uomo-macchina è stata sviluppata una Web App *Zero-Installation* in HTML5 e JavaScript. L'applicativo fa uso delle API native **Web Bluetooth** (`navigator.bluetooth.requestDevice`), applicando un filtro di discovery basato sulla proprietà `namePrefix: 'PiggyBank'`.

A causa delle differenze di *endianness* con cui i vari sistemi operativi (Windows, macOS, Android) espongono gli UUID a 128-bit ai browser, lo script implementa un array di fallback contenente tre diverse formattazioni del medesimo UUID (Ordine standard, Ordine ST invertito, Ordine GUID misto) per garantire l'accoppiamento universale.

Una volta invocato `startNotifications()`, la Web App si pone in ascolto: ad ogni notifica in ingresso dal buffer `COINS`, il parser JavaScript estrae i 5 byte e calcola in locale il saldo sommandoli in base al peso del taglio (`c10*10 + c20*20 + c50*50 + c1e*100 + c2e*200`). Per l'impostazione della soglia, la Web App converte l'input in euro inserito dall'utente applicando la formula `Math.round(euroValue * 100)` per neutralizzare gli errori di virgola mobile, per poi scrivere i byte sulla caratteristica `LIMIT` sfruttando la funzione tipizzata `DataView.setUint32(0, currentLimit, true)`.

A garanzia della sicurezza operativa, la Web App non invia attivamente l'istruzione di chiusura al servomotore: si limita a riflettere visivamente la medesima logica del firmware (`if currentTotal >= currentLimit`), demandando al microcontrollore STM32 la decisione fisica dell'interdizione dello scivolo.

5. Risultati Sperimentali

5.1 Validazione sperimentale e Telemetria

La fase di collaudo e validazione del sistema *S.C.A.L.E. PiggyBank* è stata condotta monitorando continuamente il comportamento del microcontrollore tramite l'interfaccia di telemetria e debug **USART1**. Configurato a un **baud rate di 115200 bps (8N1)**, il terminale seriale ha permesso di tracciare in tempo reale l'esecuzione del firmware cooperativo.

Al boot del sistema, l'interfaccia ha confermato il corretto superamento della fase di taratura iniziale delle celle di carico, stampando a schermo il messaggio di rito **HX711 tare completed**. A regime, il sistema ha dimostrato di poter mantenere un ciclo di aggiornamento stabile, trasmettendo ogni 500 ms una stringa riassuntiva contenente il numero di pezzi per ogni scomparto, il peso equivalente in grammi, il saldo totale in euro, la soglia limite ricevuta via BLE e lo stato logico di attivazione del servomotore.

5.2 Precisione gravimetrica e stabilità delle letture

Uno dei risultati più rilevanti ottenuti durante l'integrazione mecatronica ha riguardato la stabilizzazione delle letture analogiche. Inizialmente, la combinazione di sensori ad altissima sensibilità (HX711 a 24 bit) con una struttura in legno soggetta a micro-vibrazioni (indotte dalla caduta delle monete e dal motore di smistamento) generava fluttuazioni transitorie nei valori di peso.

L'adozione combinata di un filtro di **media mobile su 5 campioni** e di una **banda morta software di ± 2 g** ha neutralizzato con successo queste oscillazioni. I test fisici hanno dimostrato che il sistema è in grado di azzerare i falsi positivi a scomparto vuoto e di riconoscere il peso netto delle monete inserite con un margine di errore trascurabile. L'arrotondamento matematico all'intero più vicino ha garantito che, anche in presenza di fisiologiche derive termiche delle celle, il numero di monete calcolato corrispondesse sempre all'esatta quantità fisica presente nei contenitori.

5.3 Efficienza meccanica e revisione del circuito di pilotaggio

Lo smistamento geometrico passivo lungo lo scivolo in PVC ha superato i test pratici evidenziando un'efficienza di canalizzazione del 100%. Il calcolo empirico della pendenza si è rivelato esatto: con un tempo di transito di circa 2 secondi, nessuna moneta ha scavalcato la propria feritoia di pertinenza per inerzia cinetica.

Sotto il profilo elettrico, l'ostacolo più critico riscontrato nella prima fase prototipale ha riguardato il circuito di pilotaggio del motore in corrente continua dell'attuatore vibrante. Inizialmente, il progetto prevedeva di comandare l'accensione del motore direttamente dalla scheda STM32 interponendo un MOSFET di potenza. Tuttavia, i test sperimentali hanno evidenziato un limite hardware insormontabile legato alle tensioni di polarizzazione: la differenza di potenziale instaurabile tra i terminali di *Gate* e *Source* (V_{GS}) si è rivelata insufficiente.

I pin GPIO del microcontrollore, infatti, erogano una tensione logica massima di soli 3.3 V, che non permetteva al MOSFET di raggiungere la regione di piena conduzione (saturazione) necessaria a sopportare l'assorbimento di corrente del motore. D'altro canto, la linea di alimentazione a 5V disponibile sulla scheda, pur avendo una tensione adeguata, è una sorgente continua e non modulabile tramite codice: il suo utilizzo come segnale di attivazione si sarebbe tradotto in un collegamento perenne, analogo a connettere il motore direttamente a una batteria, annullando di fatto il controllo logico.

Alla luce di questa limitazione strutturale, **la decisione ingegneristica è stata quella di rimuovere del tutto il MOSFET e isolare galvanicamente l'alimentazione del motore**. Si è optato per l'impiego di un pacco batterie dedicato da 6 V azionato da un interruttore meccanico manuale.

5.4 Stabilità della connessione Bluetooth Low Energy

Il collaudo della connettività wireless ha dimostrato la perfetta tenuta del paradigma firmware "cooperativo non bloccante". Poiché il campionamento bit-banging degli HX711 disabilita gli interrupt solo per poche decine di microsecondi e non blocca mai l'esecuzione del *main loop*, il processore è riuscito a richiamare la funzione `MX_APPE_Process()` senza latenza.

Nei test prolungati di interazione con la Web App (eseguiti tramite browser Chrome abilitato alle API Web Bluetooth), la connessione GATT si è mantenuta costantemente stabile. Non sono stati registrati *timeout*, perdite di pacchetti (packet loss) o ritardi nelle notifiche periodiche dei 5 byte della caratteristica **COINS**. Anche la scrittura asincrona del parametro **LIMIT** da parte dell'utente ha innescato variazioni reattive e immediate: al raggiungimento della soglia economica, il microcontrollore ha sempre ruotato il servomotore a 90° con assoluta tempestività, bloccando lo scivolo esattamente come previsto dalle specifiche operative.

6. Conclusioni e Sviluppi futuri

6.1 Conclusioni

Il progetto S.C.A.L.E. PiggyBank ha dimostrato con successo la fattibilità, l'efficienza e l'affidabilità di un sistema mecatronico IoT a basso costo interamente gestito da un singolo System-on-Chip entry-level, l'STM32WB09. L'integrazione di tecnologie eterogenee (dalla misurazione gravimetrica analogica ad altissima risoluzione, all'attuazione meccanica PWM, fino alla connettività wireless Bluetooth Low Energy) ha dato vita a un ecosistema embedded completo e funzionale.

La sfida ingegneristica più complessa dell'intero progetto, ovvero la coesistenza in tempo reale tra le rigide tempistiche di campionamento *bit-banging* dei cinque convertitori HX711 e le stringenti latenze imposte dallo stack radio BLE 5.4, è stata superata adottando un brillante **paradigma software cooperativo non bloccante**. Attraverso lo spalmare dei cicli di taratura e di lettura mediata su più iterazioni del *main loop*, e limitando la disabilitazione degli interrupt esclusivamente al brevissimo *burst* del clock seriale, il microcontrollore ha garantito sia una reattività impeccabile nelle transazioni GATT Server, sia una precisione milligrammatica nella pesatura delle monete.

Dal punto di vista dell'architettura hardware, la transizione progettuale che ha portato a **isolare l'alimentazione e il controllo dell'attuatore vibrante** (motore DC) rispetto alla scheda logica si è rivelata una scelta ingegneristica vincente. Questa separazione ha debellato radicalmente i disturbi elettromagnetici (EMI) e le sovratensioni induttive, permettendo al sistema di raggiungere una stabilità di lettura assoluta.

Infine, sul versante dell'interazione utente (HMI), l'implementazione di una **Web App basata su API Web Bluetooth** ha chiuso l'anello di controllo eliminando la necessità di sviluppare applicazioni native, offrendo una piattaforma *Zero-Installation* reattiva e universale.

6.2 Sviluppi futuri

Sebbene il prototipo finale sia maturo e pienamente funzionante, l'analisi critica del sistema ha evidenziato alcuni margini di miglioramento ben identificabili che potranno essere oggetto di sviluppi e revisioni future:

- **Persistenza della soglia (Memoria NVM):** Attualmente, il parametro limite di risparmio viene azzerato a ogni reset o spegnimento del microcontrollore, portando il sistema a un *default* di servo aperto. Un'implementazione futura prevede il salvataggio di tale soglia nella memoria non volatile (Flash/NVM) sfruttando i moduli integrati dell'ecosistema STM32 (es. modulo `blenvm`). Questo permetterà al salvadanaio di mantenere in memoria l'ultimo obiettivo impostato dall'utente anche dopo i cicli di spegnimento.
- **Sicurezza e Autenticazione BLE:** Nella release corrente, le caratteristiche GATT sono configurate con permessi di accesso aperti (`PERM_NONE`), il che consente a qualsiasi dispositivo nel raggio di copertura Bluetooth di sovrascrivere il limite di blocco. Per l'ingegnerizzazione di un prodotto commerciale, sarà necessario implementare i protocolli di sicurezza BLE (*Pairing* e *Bonding*) affiancati a scritture autenticate, garantendo l'accesso esclusivo al legittimo proprietario.
- **Determinismo Hardware per l'Acquisizione Analogica:** Al fine di ottimizzare ulteriormente le tempistiche del *burst* di clock verso gli ADC HX711 (attualmente generato tramite ritardi software basati su istruzioni `__NOP()`), si potrà migrare l'azionamento dei pin verso un accesso diretto e a basso livello ai registri (es. `BSRR`), o delegare interamente la generazione del clock a periferiche hardware dedicate (come SPI o Timer). Questo annullerebbe il rischio di *power-down* accidentale dell'HX711 garantendo margini di *timing* perfetti.
- **Integrazione su Circuito Stampato (PCB Custom):** Il passaggio dalla prototipazione su *breadboard* a un circuito stampato dedicato consentirà di instradare in maniera ottimale i segnali, separando fisicamente i piani di massa analogici da quelli digitali per un'ulteriore riduzione del rumore termico. Tale integrazione permetterebbe anche l'aggiunta di un PMIC (*Power Management IC*) per la gestione di pacchi batteria ricaricabili al litio, innalzando la portabilità e l'efficienza del dispositivo.